INFORMATIONSBLATT

IB02 – Passwörter sicher speichern und prüfen

Dynamische Webseiten mit PHP · LS10 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie speichern Passwörter ausschließlich als Hash, prüfen eine Anmeldung mit `password_verify()` und setzen den vollständigen Login-Ablauf mit Zugriffsschutz um.

1 | Nie im Klartext

Datenbanken werden gestohlen – durch SQL-Injection, falsch konfigurierte Server oder verlorene Sicherungen. Stehen dort Passwörter im Klartext, kann der Dieb sie sofort benutzen – und weil viele Menschen dasselbe Passwort mehrfach verwenden, auch bei anderen Diensten.

Deshalb speichert man nur einen **Hash**: das Ergebnis einer Einwegfunktion. Aus dem Passwort lässt sich der Hash leicht berechnen, aus dem Hash aber nicht das Passwort.

password_hash() und password_verify() — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>password_hash</title></head>
<body>

<?php
  $passwort = "Gelato2027!";

  echo "<p>" . password_hash($passwort, PASSWORD_DEFAULT) . "</p>";
  echo "<p>" . password_hash($passwort, PASSWORD_DEFAULT) . "</p>";

  $gespeichert = password_hash($passwort, PASSWORD_DEFAULT);
  var_dump(password_verify("Gelato2027!", $gespeichert));
  var_dump(password_verify("gelato2027!", $gespeichert));
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

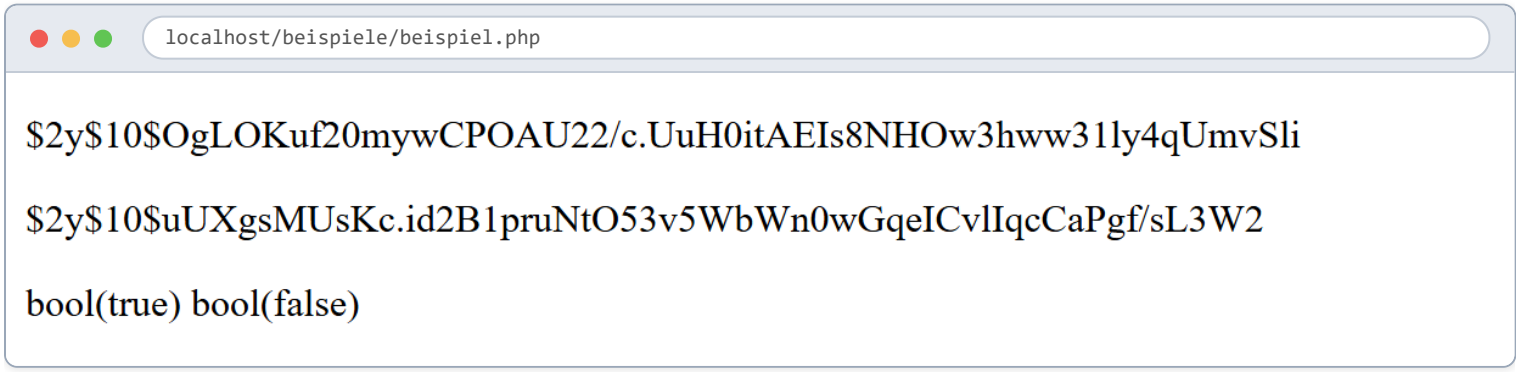


Abbildung 1: Zweimal dasselbe Passwort, zwei verschiedene Hashes – und trotzdem erkennt password_verify() das richtige Passwort.

Funktion	Aufgabe
`password_hash($pw, PASSWORD_DEFAULT)`	berechnet einen Hash (derzeit bcrypt, beginnt mit `$2y$`); ein zufälliges „Salz“ macht jeden Hash einzigartig
`password_verify($pw, $hash)`	`true`, wenn das Passwort zum Hash passt
Spalte in der Datenbank	`VARCHAR(255)` – künftige Verfahren liefern längere Hashes

> ⚠️ **NICHT SELBST BASTELN**
>
> `md5()` oder `sha1()` sind für Passwörter ungeeignet: Sie sind so schnell, dass Angreifer Milliarden Versuche pro Sekunde durchrechnen. `password_hash()` ist absichtlich langsam und immer die richtige Wahl.

2 | Der Login-Ablauf

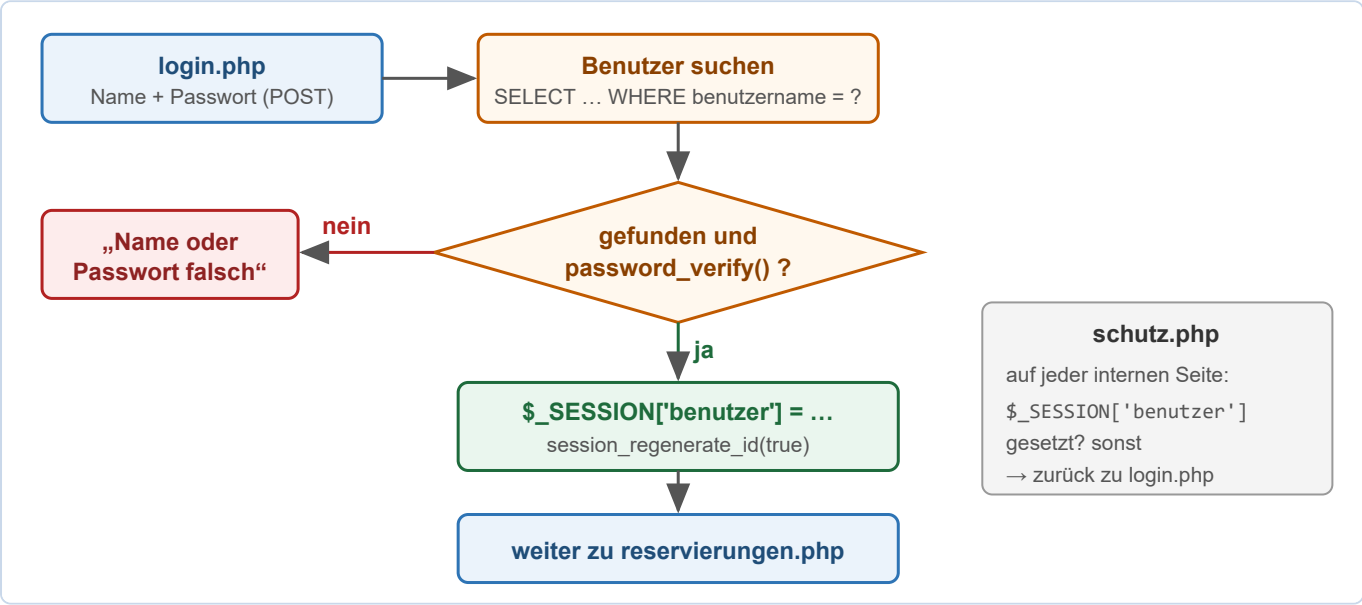


Abbildung 2: Anmeldung und Zugriffsschutz im Überblick.

login.php – Verarbeitung (oberer Teil) login.php

```php
<?php
  session_start();
  require_once 'funktionen.php';
  require_once 'daten.php';

  $fehler = '';
  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $benutzername = trim($_POST['benutzername'] ?? '');
    $passwort     = $_POST['passwort'] ?? '';

    // Benutzer suchen – natürlich mit Prepared Statement
    $stmt = $conn->prepare('SELECT passwort_hash, anzeigename FROM benutzer WHERE benutzername = ?');
    $stmt->bind_param('s', $benutzername);
    $stmt->execute();
    $benutzer = $stmt->get_result()->fetch_assoc();

    // Passwort mit dem gespeicherten Hash vergleichen
    if ($benutzer && password_verify($passwort, $benutzer['passwort_hash'])) {
      session_regenerate_id(true);                // neue Session-ID nach dem Login
      $_SESSION['benutzer'] = $benutzer['anzeigename'];
      header('Location: reservierungen.php');
      exit;
    }
    $fehler = 'Benutzername oder Passwort falsch.';
  }
?>
```

Zeile	Warum?
`prepare(… WHERE benutzername = ?)`	Der Benutzername ist eine Eingabe – SQL-Injection verhindern
`$benutzer && password_verify(…)`	Gibt es den Benutzer nicht, ist `$benutzer` `null` und die Prüfung endet sofort
`session_regenerate_id(true)`	neue Session-ID nach der Anmeldung; eine vorher untergeschobene ID wird wertlos („Session Fixation“)
eine gemeinsame Fehlermeldung	verrät nicht, ob es den Benutzernamen gibt

3 | Der Zugriffsschutz

schutz.php schutz.php

```php
<?php
// ===== Zugriffsschutz: am Anfang jeder internen Seite einbinden =====
session_start();

if (!isset($_SESSION['benutzer'])) {
  header('Location: login.php');
  exit;
}
```

reservierungen.php – erste Zeilen

```php
<?php
  require 'schutz.php';        // nur für angemeldetes Personal
  …
```

SO SIEHT ES IM BROWSER AUS

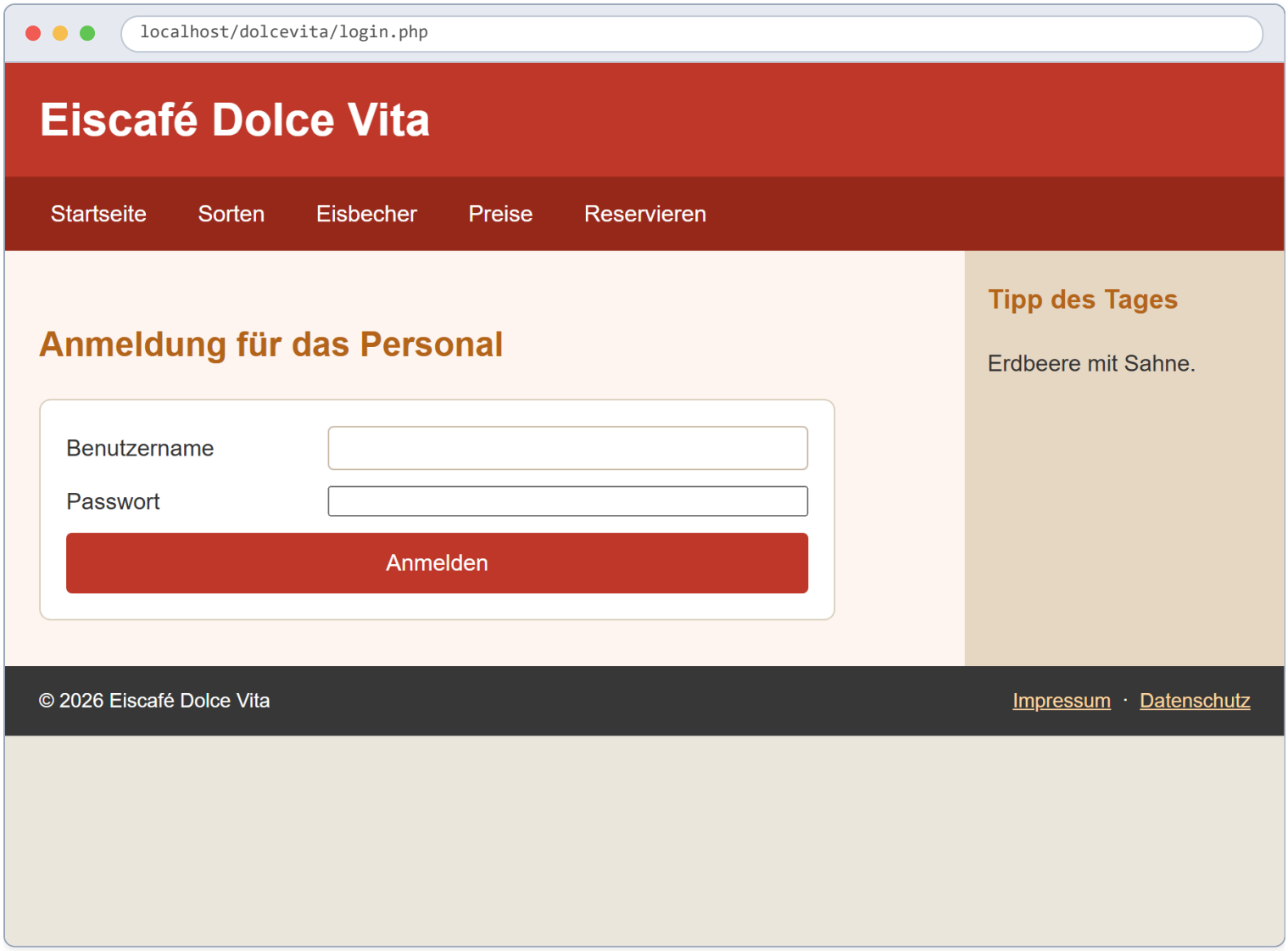


Abbildung 3: Nicht angemeldet: Der Aufruf von reservierungen.php endet auf der Anmeldeseite.

4 | Abmelden

logout.php

```php
<?php
  session_start();
  $_SESSION = [];          // alle Session-Daten löschen
  session_destroy();       // Session beenden
  header('Location: login.php?abgemeldet=1');
  exit;
```